CS 301 · MEETING 13 · RECAP 2

Solvers, Round-off, and Sparse Matrices

Operation counts, direct solves and inversion, numerical accuracy, and sparse storage.

Elimination, with loops

```python
def eliminate(A, b):
    A, b = A.astype(float).copy(), b.astype(float).copy()
    n = len(b)
    for k in range(n):                      # the unknown to remove from the rows below row k
        for i in range(k + 1, n):           # every row below row k
            factor = A[i, k] / A[k, k]      # the multiple of row k that removes the unknown from row i
            for j in range(k, n):           # every column from k on; the columns before k are already 0
                A[i, j] -= factor * A[k, j]
            b[i] -= factor * b[k]
    return A, b
```

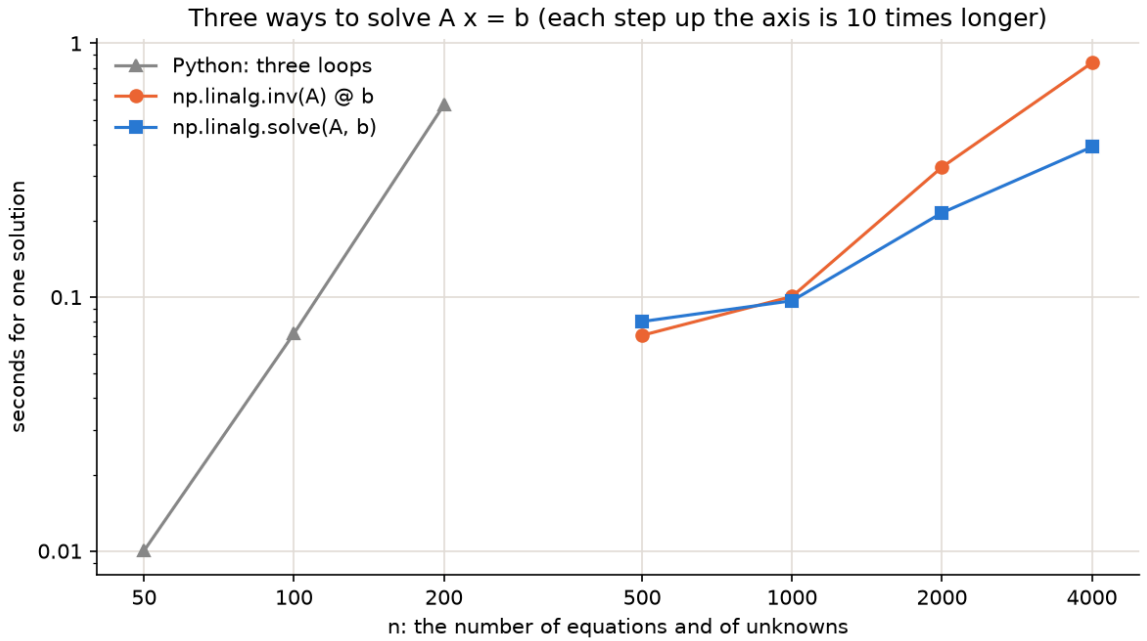
- Three nested loops: about $n^3/3$ inner iterations
- Row k is the **pivot row** of step k; `A[k, k]`, the number divided by, is the **pivot**
- Back substitution: solve from the last unknown upward

The loops, timed

n	passes, about n^3 / 3	seconds	ratio to the first	nanoseconds per pass
50	41,666	0.010	1.0	241
100	333,333	0.072	7.2	216
200	2,666,666	0.572	57.0	214

- $O(n^3)$: doubling n gives about eight times the work
- Python loops would take hours for 4,000 unknowns

Three ways to solve



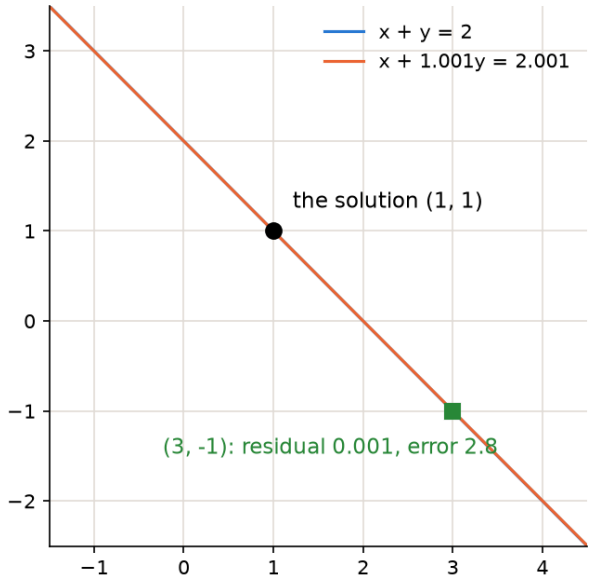
- Explicit inverse: construct all entries, then multiply by b
- `solve`: one elimination, with rows swapped where needed
- For Ax = b, prefer `solve(A, b)` to `inv(A) @ b`

Round-off: 16 digits, and the condition number

eps	condition number	error of solve	error of inverse	residual of solve	residual of inverse
1e-02	1.0e+04	4.7e-13	7.1e-12	1.1e-16	3.2e-13
1e-06	1.0e+08	6.7e-09	9.9e-08	2.1e-16	1.0e-08
1e-10	1.0e+12	5.3e-05	4.1e-03	2.0e-16	1.6e-04
1e-14	1.0e+16	6.3e-01	1.9e+01	3.9e-16	1.0e+00

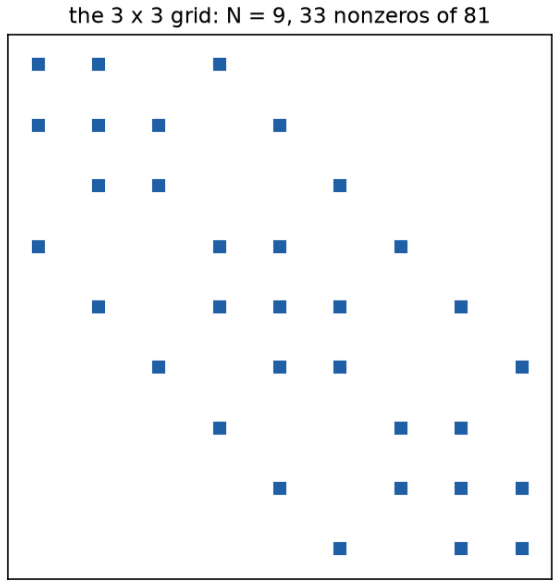
- Potential digits ≈ $\max(0, 16 - \log_{10}(\text{condition number}))$
- Here, `solve` has a residual near 10^{-16}

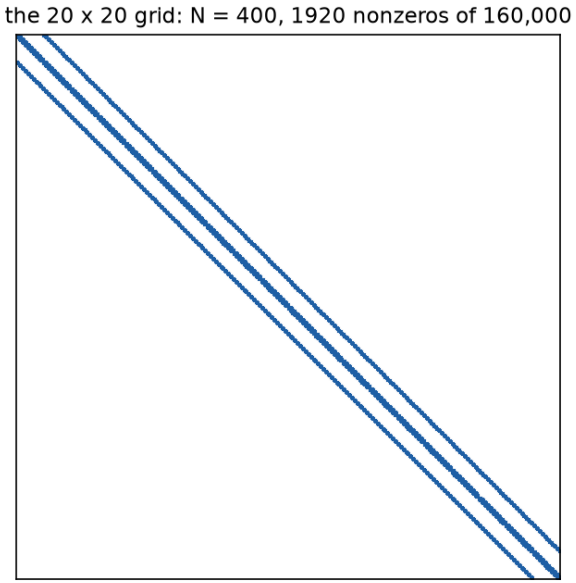
A small residual can accompany a large error



- A point that nearly satisfies both equations can be far from the solution
- Assess the residual together with the condition number

Mostly zeros: the grid





- $N = n^2$ unknowns; at most five nonzeros per row
- 100×100 grid: 100 million entries, about 50,000 nonzeros

COO, CSR, CSC

```
the 3 x 3 grid: 33 nonzeros; the first 12 entries of each array
COO  row      0  0  0  1  1  1  1  2  2  2  3  3 ...
     col      0  3  1  1  4  0  2  2  5  1  3  0 ...
     data     4 -1 -1  4 -1 -1 -1  4 -1 -1  4 -1 ...
CSR  indptr   0  3  7 10 14 19 23 26 30 33   where each of the 9 rows begins; the last entry is 33
     indices  0  1  3  0  1  2  4  1  2  5  0  3 ...
     data     4 -1 -1 -1  4 -1 -1 -1  4 -1 -1  4 ...
CSC  indptr   0  3  7 10 14 19 23 26 30 33   where each column begins
     indices  0  1  3  0  1  2  4  1  2  5  0  3 ...
```

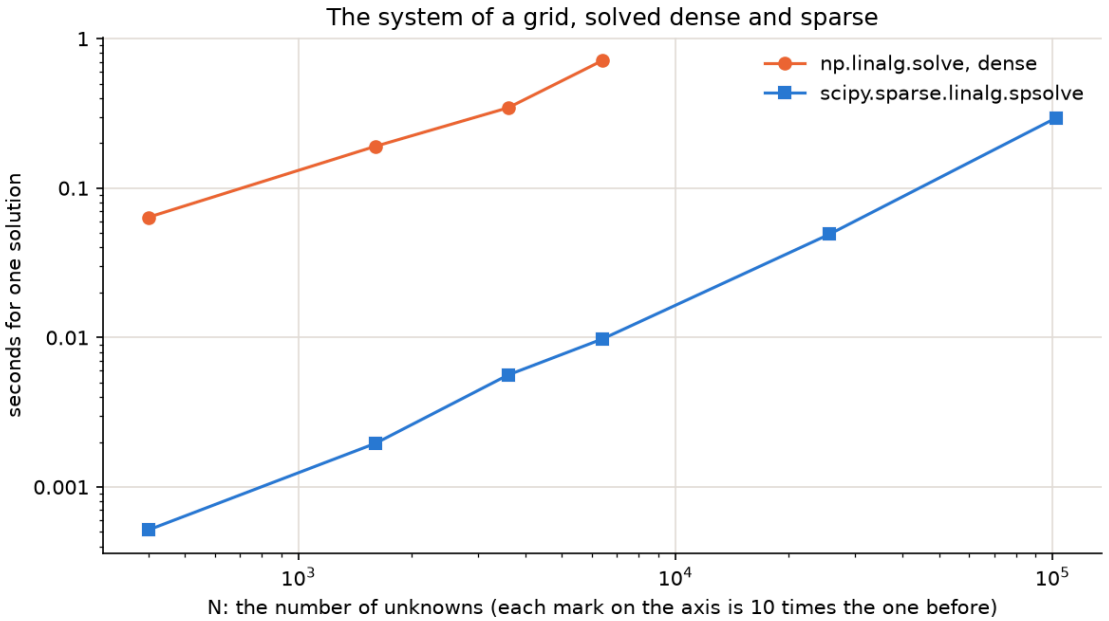
- **COO**: row, column, value of each nonzero; easy to build
- **CSR**: values and columns row by row, and `indptr`, where each row begins; for `A @ v`
- **CSC**: the same by columns; for solvers

Memory: 8 bytes a number, or 12 bytes a nonzero

```
    grid  N = unknowns    nonzeros   dense, 8 bytes a number   CSR, 12 bytes a nonzero
   10 x 10            100         460                    80 KB                  5.92 KB
   30 x 30            900       4,380                  6.48 MB                  56.2 KB
  100 x 100        10,000      49,600                   800 MB                   635 KB
  300 x 300        90,000     448,800                  64.8 GB                  5.75 MB
 1000 x 1000    1,000,000   4,996,000                     8 TB                    64 MB
 3000 x 3000    9,000,000  44,988,000                   648 TB                   576 MB
check on the 100 x 100 grid, built: 49600 nonzeros
```

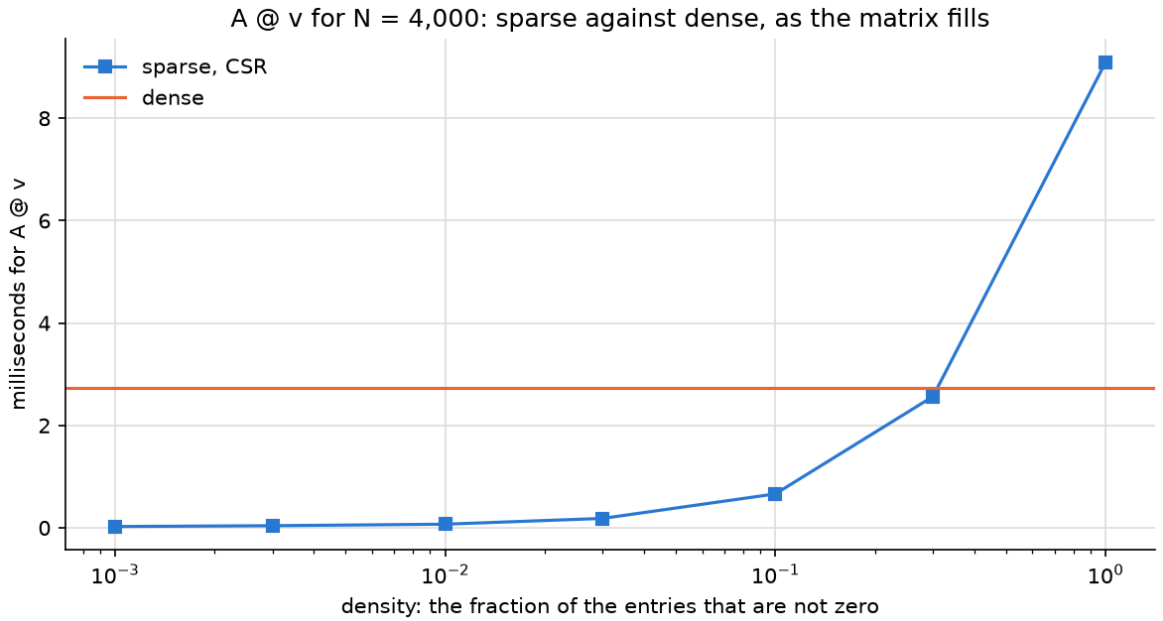
- For this grid: dense storage grows as N^2, sparse as N
- The 1,000 × 1,000 grid: 8 TB dense, 64 MB sparse

Solving, dense and sparse



- Dense: $O(N^3)$ work; larger benchmarks omitted for memory use
- Sparse: slower growth on this grid, tested to N = 102,400

The cost of a product: O(nnz)



- Dense: N^2 multiply-adds; CSR: one per stored nonzero
- Sparse access also reads indices and scattered entries of v
- Grid density ≈ 5 / N; compare with the measured crossover

What a system costs

	stored	solved	multiplied by a vector
dense, N unknowns	$8N^2$ bytes	$O(N^3)$	$O(N^2)$
sparse, nnz nonzeros	12 nnz + 4(N+1) bytes	depends on fill-in	O(nnz)

- Conditioning limits attainable accuracy
- Fill-in affects the cost of sparse solving